Copyright All rights reserved

10. 열거형
(enumeration)

Heesung Oh
https://github.com/3dapi

10. 열거형(enum)

Heesung Oh
Copyright All rights reserved

- 심볼릭 상수
  - 값 대신 의미 있는 이름 사용
  - 전처리문
    - 전처리 단계에서 단순 치환
    - 메모리 사용 없음
    - 타입 없음

```
#define MAX_HP          1000
#define STATE_ATTACK    2
```

  - const (C / C++)
    - 읽기 전용 변수, 타입 존재, 메모리 사용
    - C++: 컴파일 타임 상수
    - C: 수정 불가능한 변수. 상수 표현식 아님
      (배열 크기, case 레이블 사용 불가)
- 열거형
  - 이름으로 묶인 정수 값 그룹. 사용자 정의 자료형

10.1 열거형(enum) 개념과 특징

Heesung Oh
Copyright All rights reserved

- **열거형**
  - **이름을 묶인 정수 값 그룹, 사용자 정의 자료형**
- **열거형 선언**

```
enum 열거형_태그
{
    열거자1,                // 열거자만 선언
    열거자2 = 0x10,         // 열거자와 함께 값을 함께 선언
    열거자3,                // 앞 열거자 항목의 값에서 1 증가
    열거자4 = 0x10,         // 같은 값을 가진 열거자
    // 열거자x = 3.14,      // 정수형만 허용
    열거자6 = (int)3.14     // 컴파일 시점에서 캐스팅
    ...,
};                          // 끝에 반드시 세미콜론을 넣는다.
```

- **열거형 사용**

```
enum 열거형_태그 변수 이름 = 열거자...;
```

10.1 열거형(enum) 사용 예

Heesung Oh
Copyright All rights reserved

- **열거형 선언**

```
enum CharacterState
{
    STATE_IDLE,      // 0
    STATE_MOVE,      // 1
    STATE_ATTACK,    // 2
    STATE_DEATH      // 3
};
```

- **열거형 사용**

```
// 사용할 때 enum 열거형_태그 변수 이름
enum CharacterState state = STATE_ATTACK;
```

10.1 열거형(enum) 사용 예

Heesung Oh
Copyright All rights reserved

```
enum COLOR_RGBA
{
  COLOR_RED,
  COLOR_GREEN,
  COLOR_BLUE,
  COLOR_ALPHA,

  COLOR_RGBA_END = COLOR_ALPHA,
  COLOR_TEMP     = 100,
  COLOR_OTHER    = 100,

  COLOR_MAX = COLOR_ALPHA + 1,

  COLOR_B  = 'B',
  COLOR_C,

  COLOR_PI = (int)3.14,
};
```

```
COLOR_RED   → 0
COLOR_GREEN → 1
COLOR_BLUE  → 2
COLOR_ALPHA → 3

COLOR_TEMP  = 100
COLOR_OTHER = 100   // 같은 값 허용

COLOR_MAX = COLOR_ALPHA + 1   // 3+1=4

COLOR_B = 'B'   // 'B' → 아스키 코드 66
COLOR_C         // 67 (COLOR_B + 1)

COLOR_PI = (int)3.14   // 3
```

10.2 이름 없는 열거형 (Anonymous Enum)

Heesung Oh
Copyright All rights reserved

- 이름없는 열거형
  - enum과 동일 문법
  - 태그(Tag) 없음
  - 자료형 생성 없음
    - 타입 생성 안함
    - 변수 선언 불가
    - 상수 집합 전용

- 목적
  - 관련 상수 그룹화
  - 지역 범위 상수 정의
    - → 매직 넘버 제거 → 의미 명확, 가독성 향상

10.2 이름 없는 열거형 특징

Heesung Oh
Copyright All rights reserved

- 컴파일 시점 상수
  - 정수 상수 → 실행 중 변경 불가
  - 배열 크기 사용 가능
- 유효 범위 (Scope)
  - 블록 / 함수 내부 사용 가능 ← 선언된 범위 내에서만 유효
  - struct 내부 사용 비권장, ※C++ 에서 가능
  - 구조체 종속 상수로 사용 불가
- 사용 시 주의
  - 타입 기반 설계 불가
- 다음 경우 태그 있는 enum 사용
  - 상태(State) 표현
  - 함수 인자 / 반환값
  - 타입 구분 필요한 경우

10.2 Anonymous Enum 활용

Heesung Oh
Copyright All rights reserved

- 1) 상수 그룹화 및 배열 크기 정의

```
enum
{
    MAX_LEVEL = 99,
    MAX_SKILLS = 10,
    INVENTORY_SLOTS = 30
};

int skillLevels[MAX_SKILLS];     // 컴파일 타임 상수
int inventory[INVENTORY_SLOTS];
```

10.2 Anonymous Enum vs 심볼릭 상수

Heesung Oh
Copyright All rights reserved

- 심볼릭 상수 (#define)
  - 전처리 단계 치환
  - 타입 없음
  - 타입 검사 불가
  - 디버깅 어려움
- const
  - 읽기 전용 변수
  - 타입 존재
  - C89/C90 기준 → 컴파일 타임 상수 아님
  - 배열 크기 사용 불가
- 이름 없는 열거형 (Anonymous Enum)
  - 컴파일 시점 상수
  - 타입 검사 가능
  - 배열 크기 사용 가능

10.2 Anonymous Enum vs 심볼릭 상수

Heesung Oh
Copyright All rights reserved

구분	#define 매크로	const 변수	이름 없는 열거형
처리 단계	전처리기 (치환)	컴파일러 (변수)	컴파일러 (상수)
타입 체크	불가능	가능	가능
디버깅 심볼	확인 어려움	확인 가능	확인 가능
배열 크기 사용	가능	불가능 (표준 C)	가능

10.3 열거형(enum)과 typedef

Heesung Oh
Copyright All rights reserved

- typedef → 자료형 별칭
  - 이름 단순화
  - 가독성 향상
- enum 사용 방식
  - enum 반복 사용 → 코드 길어짐
- typedef 사용
  - enum 키워드 생략 가능

10.3.1 enum, typedef

Heesung Oh
Copyright All rights reserved

- **enum + typedef 분리**

```
enum CharacterState
{
    // typedef로 열거형의 별칭 생성
};

// typedef로 열거형의 별칭 생성
typedef enum CharacterState CharacterState;
```

- **열거형 태그를 자료형처럼 사용**

```
CharacterState state = STATE_IDLE;
```

10.3.2 enum, typedef 한 번에 선언 (권장)

Heesung Oh
Copyright All rights reserved

- enum + typedef 동시 선언 (권장)
  - 가장 일반적인 방식
  - 실무에서 표준 패턴

```
typedef enum MyCharacterState(생략가능)
{
    STATE_IDLE,
    STATE_MOVE,
    STATE_ATTACK,
    STATE_DEATH
} MyCharacterState, OtherCharacterState;
```

선언 방식	특징
enum 먼저 선언 → typedef 별칭 추가	기존 enum에 이름을 추가할 때 유용 (외부 라이브러리 등)
typedef enum { … } TypeName;	가장 일반적이며 실무·교재에서 주로 사용

10.4 열거형 사용: 상태 코드 정의하기

Heesung Oh
Copyright All rights reserved

- 상태 표현에 적합 → 숫자 대신 의미 있는 이름용 → 가독성 향상

```
typedef enum CharacterState
{
    STATE_IDLE,       // 대기
    STATE_MOVE,       // 이동
    STATE_ATTACK,     // 공격
    STATE_HIT,        // 피격
    STATE_DEATH       // 사망
} CharacterState;

//사용 예:
CharacterState state = STATE_IDLE;

void ChangeState(CharacterState newState)
{
    state = newState;
}
```

```
typedef enum ResultCode
{
  RESULT_OK,              // 성공
  RESULT_ERROR_FILE,      // 파일 에러
  RESULT_ERROR_MEMORY,    // 메모리 부족
  RESULT_ERROR_UNKNOWN    // 알 수 없는 에러
} ResultCode;

ResultCode LoadData(const char* filename)
{
    // 파일을 열고 데이터를 읽는다고 가정
    // 상황에 맞게 적절한 코드 반환
    return RESULT_OK;
}
```

```
ResultCode result = LoadData("data.txt");

if (result != RESULT_OK) {
    printf("데이터 로드 실패! 에러 코드:
%d\n", result);
}
```

10.5 열거형과 switch-case 활용

Heesung Oh
Copyright All rights reserved

- enum → 상태 정의
- switch → 상태 분기
- 함께 사용 → 코드 구조 명확

- if - else 방식
  - if (state == STATE_IDLE) { … }
  - else { … }
  - 조건 반복, 가독성 낮음

- switch - case 방식
  - switch (state) {
  - case STATE_IDLE: …
  - case STATE_MOVE: …
  - 구조 명확, 상태 구분 쉬움

10.5 열거형과 switch-case 활용 예

Heesung Oh
Copyright All rights reserved

```c
typedef enum CharacterState
{
  STATE_IDLE,
  STATE_MOVE,
  STATE_ATTACK,
  STATE_DEATH
} CharacterState;
```

```c
void UpdateCharacter(CharacterState state)
{
  switch (state)
  {
    case STATE_IDLE:
      printf("캐릭터가 가만히 서 있습니다.\n");
      break;
    case STATE_MOVE:
      printf("캐릭터가 이동 중입니다.\n");
      break;
    case STATE_ATTACK:
      printf("캐릭터가 공격합니다!\n");
      break;
    case STATE_DEATH:
      printf("캐릭터가 쓰러졌습니다.\n");
      break;
    default:
      printf("알 수 없는 상태입니다.\n");
      break;
    }
}
```